

人工智能基础与应用 · 大模型实训

大模型实训讲义 · 全书

从零实现大模型：M0-M9 知识主干（16 周 / 11 课题 / 零预算）

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

Markdown 母版：人工智能基础与应用/讲义/ 下的 .md 文件 · 配套上游：CS336 A1-A5 · nanochat · minimind

版权与使用说明

这是什么。本 PDF 是「人工智能基础与应用·大模型实训」知识轨的讲义合集，由讲义/*.md 自动排版生成。讲义与课题一一对应：每讲都由「要回答的问题 → 主干 → 伪代码骨架 → 代码在哪里 → 常见误解 → 能立刻跑的实验 → 代码级提问 → 记忆锚」八件套组成。**为什么以 Markdown 为母版。**PDF 便于通读与打印，但唯一真相是仓库里的 .md：内容修订、代码定位、实验数据只改 Markdown，PDF 随时重新生成。看到 PDF 与仓库不一致时，以仓库为准。**重新生成。**

`cd 人工智能基础与应用`

`环境/pdf/生成讲义PDF.sh --book` # 生成全书

`环境/pdf/生成讲义PDF.sh --all` # 每讲各出一个 PDF

字体与排版。正文思源宋体 (Noto Serif SC)，标题思源黑体 (Noto Sans SC)，代码 DejaVu Sans Mono；三者均按 SIL OFL 1.1 / Bitstream Vera 许可授权，可自由再分发。排版引擎 typst 0.15。字体文件由 `环境/pdf/bootstrap.sh` 从 npm (@expo-google-fonts 系列) 取得，编译过程完全离线。

读法建议。先读每讲开头的「要回答的问题」，自己试着答一遍；再读对标名师的原始材料（见《对标教学资源清单》）；然后回到本讲义看主干与代码定位；最后做「代码级提问」并写下预测，再动手改代码验证。预测错不是失败——**预测错 + 实验打脸**才是这套学法里质量最高的时刻。

目录

版权与使用说明	2
目录	3
讲义库·使用方法（三师制里的“我”那一腿）	5
讲义规范（每讲必含 8 件）	5
📖 下载 PDF（离线阅读 / 打印用）	5
目录与状态	5
使用流程（照做）	6
讲义 M0·数学与机器学习地基	7
○、本讲要回答的 5 个问题	7
一、自动微分：为什么反向传播只要一遍	7
二、信息量、交叉熵、困惑度、bpb（跨分词器可比性的根源）	7
三、算力与显存记账（估数量级是核心能力）	8
四、数值稳定性：softmax 与 logsumexp	8
五、“loss 不降”的排查顺序（把知识变成动作）	9
六、能立刻跑的实验	9
七、代码级提问（3-5 道）	9
八、记忆锚	10
讲义 M1·从 n-gram 到 Transformer	11
○、本讲要回答的 5 个问题	11
一、n-gram：用计数做语言模型	11
二、词向量：从 one-hot 到分布式表示	11
三、RNN / LSTM：把“历史”压进一个向量	11
四、seq2seq 与注意力（2014）：注意力的真正起源	12
五、常见误解	12
六、能立刻跑的实验	12
七、代码级提问（3-5 道）	13
八、记忆锚	13
讲义 M2·分词与表示	14
○、本讲要回答的 5 个问题	14
一、压缩视角：BPE 到底在做什么	14
二、为什么必须字节起步（以及代价）	14
三、训练算法：从 $O(n^2)$ 到官方要求的 <1.5 秒	15
四、编码：回放合并历史，而不是最长匹配	15
五、接口设计决定内存上界（encode vs encode_iterable）	15
六、词表大小的三方权衡（第 1、4 题的答案框架）	16
七、常见误解（每条配反例）	16
八、能立刻跑的实验（讲完就做，知行咬合点）	16
九、代码级提问（3-5 道，先写预测再动手）	17

十、记忆锚	17
十一、本讲与后续的接口（预告）	17
讲义 M3 · Transformer 内部机制	18
○、本讲要回答的 5 个问题	18
一、注意力：加权检索 + 为什么缩放 $\frac{1}{\sqrt{d_k}}$	18
二、多头：拆开看不同关系，但总算力不变	18
三、位置编码：从“贴标签”到“转角度”	19
四、归一化与残差：让 100 层能训起来的两件套	19
五、FFN、参数量与 KV cache（把结构换算成资源）	20
六、常见误解（每条配反例）	20
七、能立刻跑的实验与改坏实验（课题 02 的里程碑）	20
八、代码级提问（3-5 道）	21
九、记忆锚	21
讲义 M4-M9 · 展开大纲（按周写完整版）	22
M4 · 训练：数据、优化、系统（W3-W9）	22
M5 · 推理与部署（W5-W8）	22
M6 · 后训练：从“会说话”到“会做事”（W10-W12）	22
M7 · 评测：把“感觉变好了”变成“确实变好了”（W9-W12）	23
M8 · 智能体：工具、记忆、harness、长程任务（W13-W14）	23
M9 · 前沿与安全（W15-W16 选题制）	23
展开时机表	24

讲义库 · 使用方法（三师制里的“我”那一腿）

立库 2026-09-17。为什么要有这个库：用户指出“实操性够强，但知识讲解偏少”。《知识地图》回答“学什么”，《对标资源清单》回答“跟谁学”，本库回答“此刻这一讲到底讲了什么”——由我按你的水平写的主讲义。三者关系不是替代，是分工：

- 对标名师（Raschka / 李沐 / 李宏毅 / CS336）= 权威原始讲法，读它打地基
- 本库讲义 = 为你量身写的浓缩主干 + 与代码的对应 + 常见误解（省你时间）
- 我（对话里）= 只讲你卡住的那一层，出代码级提问，判你的理解

讲义规范（每讲必含 8 件）

#	部件	作用
1	本讲要回答的 3-5 个问题	带着问题读，读完能答才算过
2	主干讲解（≤150 字/概念块）	一次讲透，不挤牙膏
3	公式/伪代码骨架	这是“手写区”的地图， 不是答案
4	在代码里长什么样	指向手写/或上游 nanochat 的 函数与接口 （不给行号、不给实现）
5	常见误解（每条配一个反例）	抄自历届踩坑与官方测试的隐含要求
6	能立刻跑的实验	讲完就跑，知行咬合点
7	代码级提问（3-5 道）	改坏它/预测现象/给出诊断
8	记忆锚（隐喻 + 口诀 + 符号位置）	进 drill-ledger

📖 下载 PDF（离线阅读 / 打印用）

不用看屏幕读 Markdown——讲义已排成 PDF，直接下载：

下载这个	页数	说明
讲义/pdf/大模型实训讲义-全书.pdf	24	★ 首选：封面 + 版权说明 + 目录 + M0-M3 + 大纲
讲义/pdf/M0...M3.pdf 等单讲分册	2-5	手机上读、单讲打印、发给同学
章程与地图/pdf/章程与地图-全书.pdf	28	九份章程合订（学什么/跟谁学/什么叫学过/何时学/谁写代码）

排版与验收：正文思源宋体、标题黑体、代码等宽；全书 24 页里 99 条去重公式已**逐条单独编译验证**过，每讲章节也做了齐备性自检（防排版吞内容）。重生成与维护见 `环境/pdf/README.md`。

```
cd 人工智能基础与应用 && 环境/pdf/生成讲义PDF.sh # 每讲 + 全书
cd 人工智能基础与应用 && 环境/pdf/生成讲义PDF.sh --chapter # 章程分册
```

铁律：母版是这里的 `.md`，PDF 是派生物。改了讲义必须重跑一次生成脚本。

目录与状态

讲义	模块	对应课题/周次	状态	PDF
M0-数学与机器学习地基.md	M0	贯穿, W1 起	✓ 已写	✓ 5 页
M1-从n-gram到Transformer.md	M1	课题 02, W2	✓ 已写	✓ 4 页
M2-分词与表示.md	M2	课题 01, W1 (当前)	✓ 已写	✓ 5 页
M3-Transformer内部机制.md	M3	课题 02, W2-W3	✓ 已写	✓ 5 页

讲义	模块	对应课题/周次	状态	PDF
M4-M9-待展开大纲.md	M4-M9	W4-W16	🕒 大纲已就位，按周展开	✓ 4 页

PDF 生成日期：2026-09-17（母版有改动就要重跑 环境/pdf/生成讲义 PDF.sh）。

展开节奏：**每周讲之前，我把当周讲义写完整**（不留空讲义占位）；本文件表格同步更新状态。

使用流程（照做）

- ① 预习：读本讲“要回答的问题”（2 分钟）
- ② 读对标材料对应章节（≤25 分钟，带着问题读）
- ③ 读本库讲义（≤15 分钟，看主干与“代码里长什么样”）
- ④ 合上讲义，用自己的话讲给我听 → 我专挑含糊处追问
- ⑤ 进手写区（R 类任务）→ 跑判分
- ⑥ 做“改坏实验”→ 先预测 → 再跑 → 修好
- ⑦ 我出代码级提问，你作答 → 进 drill-ledger 建卡

纪律：讲义里**不含可直接粘贴到手写/的实现**（见《造轮子边界.md》）。讲义给的是“坐标系 + 陷阱 + 判据”，代码必须你自己长出来。

讲义 M0 · 数学与机器学习地基

对应：贯穿全程，W1 起 | 前置讲义，不单独占周次，随用随补 对标材料：李沐 d2l ch2–ch5 + ch11（优化）· Karpathy micrograd 视频 · probability/ 课题 01–08（已建成，直接搭桥）为什么先讲这个：后面所有的“为什么训练崩了”“为什么显存不够”“为什么换分词器 loss 不能比”，答案都在这一讲里。它不是复习，是工具箱。

〇、本讲要回答的 5 个问题

1. 梯度是怎么穿过 100 层网络的？为什么反向传播只要一遍？
2. 交叉熵、困惑度、bpb 三者的关系是什么？为什么跨模型只能比 bpb？
3. 训练一个 N 参数的模型，显存和算力到底怎么估？
4. 为什么 softmax 要先减最大值？不这样做会怎样？
5. “loss 不降”时，我该按什么顺序怀疑？

一、自动微分：为什么反向传播只要一遍

计算图：把 $f(x_1, \dots, x_n)$ 表示为有向无环图，每个节点是一次基本运算。链式法则：若 $y = f(g(x))$ ，则 $\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$ 。

两种方向的差别（这是本讲的第一个关键直觉）：

模式	做法	成本	适合
前向模式	从输入往输出推导数	每个输入要一遍 → n 遍	输入少、输出多
反向模式	从输出往输入推导数	一遍拿到全部输入的偏导	输入多、输出少 ← 我们的情况

我们的情况是“几亿个参数（输入多）→ 一个标量 loss（输出少）”，所以反向模式（就是我们说的反向传播）是**首选**：1 次前向 + 1 次反向 = 全部梯度。

梯度累加的工程含义：反向时每个节点把上游梯度**相加**。这解释了两个常见现象：①同一个张量被用了两次（如权重共享），梯度会累加；②既然梯度是“累加”的，就能**梯度累积**（gradient accumulation）——多个小批次的梯度加起来等价于大批次，这是后来显存不够时的核心手段（课题 04 会用到）。

在代码里长什么样：

- 手写/里会有一个 Value 类（或等价的张量自动微分），核心是 `.backward()`：按**拓扑逆序**遍历，每个节点用局部导数乘上游梯度累加到父节点。
- 上游对照：nanocat/optim.py 里参数更新前的 `p.grad` 就是这一机制的产物；CS336 A1 的 `run_gradient_clipping` 则是在使用梯度（先要能算出来）。

二、信息量、交叉熵、困惑度、bpb（跨分词器可比性的根源）

信息量：一个概率为 p 的事件携带的信息是 $-\log p$ （nats，底为 e ；若底为 2 则单位是 bit）。

熵：随机变量的平均信息量 $H(p) = -\sum_x p(x) \log p(x)$ 。直觉：“平均每个符号有多难猜”。

交叉熵：用模型分布 q 去编码真实分布 p 的代价

$$H(p, q) = -\sum_x p(x) \log q(x)$$

语言模型里 p 是“真实下一个 token”， q 是模型输出——这正好就是训练 loss。

困惑度： $\text{PPL} = \exp(H(p, q))$ (nats 为底时)。直觉最有用的一句话：“模型平均在多少个等概率选项里犯迷糊。” $\text{PPL}=10$ 相当于“平均 10 选 1”。

bpb (bits per byte)：

$$\text{bpb} = \frac{H(p, q) [\text{nats}]}{\ln 2 \times \# \text{bytes per token}}$$

分母是“平均每个 token 对应多少字节”。

为什么必须用 bpb 跨模型比较：交叉熵的单位是“每 token”，而 token 的大小随分词器变化——词表越大 token 越“大”，同样文本的 token 数越少，单 token 交叉熵天然越高。除以“每 token 字节数”就把记账单位统一到“每个字节”上，于是与分词器无关。这也解释了 M2 讲义的结论：换词表后 loss 不可比，bpb 可比。

在代码里长什么样：nanochat/loss_eval.py 专门做 bpb（而不是 loss）的评估；nanochat/core_eval.py 则是另一类指标（CORE，由 22 个数据集的准确率合成）。你在课题 01 的压缩率表，其实是在预演 bpb 的分母——先感受“每 token 多少字节”。

三、算力与显存记账（估数量级是核心能力）

参数量（Transformer 主项，细节见 M3）：

- 每层注意力 $4d^2$ （Q/K/V/O 四个 $d \times d$ ），MLP 约 $8d^2$ （SwiGLU 的等价形式）→ 每层 $\approx 12d^2$
- 词嵌入 $V \cdot d$ （权重共享时 1 份）

算力（训练）：业界通用估算

$$\text{总 FLOPs} \approx 6 \times N \times D$$

其中 N 是非嵌入参数量， D 是训练 token 数。拆解：每个参数在前向贡献 2 FLOPs（乘+加），反向是前向的 2 倍 → 合计 6。注意力部分另有 $12Lhqt$ 项（序列长度相关，短序列时可忽略）。

Karpathy 在 nanochat 文档里给的就是这个公式，并特别注明“与 Chinchilla 略有 1% 差异，因为不计嵌入与 softmax”。

显存（训练）：粗算每参数

项	字节数
参数 (bf16)	2
梯度 (bf16)	2
Adam 一阶动量 (fp32)	4
Adam 二阶动量 (fp32)	4
小计（不含激活、不含 fp32 主权重）	$\approx 12-16$
激活（随 batch $\times$ 序列长度增长，可用 checkpointing 换）	视配置

练习式直觉：0.1B 参数 $\times 16$ 字节 $\approx 1.6\text{GB}$ （优化器+梯度+权重）；剩下的全是激活与临时张量——这就是为什么 16GB 的 T4 训 0.1B 要精打细算（课题 04 的真实约束）。

在代码里长什么样：nanochat/gpt.py 里有 estimate_flops() 与 num_scaling_params() 两个方法，正是上面两段公式的代码化。课题 03 的 scaling law 就是靠这两个函数把“跑多大”换算成“花多少算力”的。

四、数值稳定性：softmax 与 logsumexp

问题： $\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$ 。若某个 $x_i = 1000$ ， e^{1000} 直接溢出成 inf。

解法：分子分母同乘 e^{-m} ($m = \max_j x_j$):

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

数值上完全等价，但最大指数变成 $e^0 = 1$ ，永不溢出。“减最大值”不是技巧，是恒等变形。

logsumexp 技巧: $\log \sum_j e^{x_j} = m + \log \sum_j e^{x_j - m}$ ，这是所有“用 log 概率做加法”的算法（交叉熵、CTC、HMM、flash attention 的 online softmax）的共同基础。

为什么这很重要：课题 05 的 **online softmax** 就是“分块计算时如何边算边保持数值稳定”，其思想内核与这里的减最大值完全一致——你在 M0 学的东西，会在 Triton kernel 里原样再出现一次。

五、“loss 不降”的排查顺序（把知识变成动作）

1. **数据**：label 与输入错位了吗？分词是否正常（看打印出来的样本）？
2. **初始化**：全零/全同初始化会让对称性无法打破；检查 std 与层数是否匹配
3. **学习率**：太大 → loss 震荡/NaN；太小 → 曲线是平的。先用小数据能否过拟合一条样本自检
4. **形状与 mask**：mask 是否在 softmax 之前施加？是否广播到正确维度？
5. **数值**：是否出现 inf/nan？打开梯度范数监控
6. **数值精度**：bf16 在小模型/小梯度上可能不够（T4 没有 bf16，课题 04 要用 fp16 + GradScaler）

这份顺序本身就是“知行合一”：它将成为课题 04 的排查清单，而不是一段可背诵的文字。

六、能立刻跑的实验

```
# ① 手算 bpb 的换算（先自己推，再看输出对不对）
环境/.venv/bin/python - <<'PY'
import math
# 假设某模型：每 token 交叉熵 2.0 nats，平均每个 token 对应 4 个字节
loss_nats, bytes_per_token = 2.0, 4.0
ppl = math.exp(loss_nats)
bpb = loss_nats / math.log(2) / bytes_per_token
print(f"loss={loss_nats} nats → PPL={ppl:.2f}, bpb={bpb:.3f}")
print("→ 再推一遍：若词表翻倍使 bytes/token 变成 6，bpb 会怎么变？loss 呢？")
PY

# ② 验证“减最大值”是恒等变形（含极端值）
环境/.venv/bin/python - <<'PY'
import numpy as np
x = np.array([0.0, 1000.0, 1000.0])
naive = np.exp(x) / np.exp(x).sum()      # 溢出
stable = np.exp(x - x.max()) / np.exp(x - x.max()).sum()
print("naive :", naive, " ← 溢出成 nan")
print("stable:", stable, " ← 正确")
PY
```

七、代码级提问（3–5 道）

1. **【推导】** 用 e^{-m} 的恒等变形，证明“减最大值”不改变 softmax 的输出（写出两步）。

2. 【估算】 一个 0.1B 参数模型用 Adam 训练 (bf16 参数 + fp32 优化器状态): ①权重+梯度+优化器状态占多少 GB? ②若 T4 只有 16GB, 留给激活的预算是多少?
 3. 【预测】 训练中把 loss 从 fp32 计算改成 fp16 计算: 训练会更快还是更慢? 会出现什么新故障? (提示: T4 无 bf16; GradScaler 是干什么的)
 4. 【联系】 probability/ 里学过的“期望”和这里的“交叉熵”是同一个东西的两种说法吗? 试着用期望的符号把交叉熵写出来。
 5. 【诊断】 曲线在 400 步突然 NaN。按本讲 § 五 的顺序, 你会先看哪三个量?
-

八、记忆锚

- 隐喻: 反向传播 = “责任回溯”——loss 是最终事故, 每个参数按“它贡献了多少”分摊责任; 责任可以累加, 所以梯度能攒 (梯度累积)。
- 口诀: “一遍反向、六倍算力、十六字节、先减最大值” (反向传播一遍 / 6ND / 每参数约 16 字节 / softmax 稳定性)
- 符号位置法: 把 $\text{bpb} = \frac{H}{\ln 2 \cdot B}$ 写在卡片中央, 分子 H 是“每 token 的惊讶”, 分母 B 是“每 token 的字节”, 中间一条分数线 = “把单位从 token 换成 byte”——这条线就是跨分词器比较的桥。

讲义 M1 · 从 n-gram 到 Transformer

对应课题：课题 02 (W2) 前半 | 模块：M1 | 预计 1 个时段 对标材料：李沐 d2l ch8–ch10 (RNN → 注意力) · CS336 lecture 01 (概览) · 《Attention Is All You Need》原文 (走论文精读) **为什么讲这段**：不理解 attention 之前的世界，就不知道它在解决什么问题——而“知道它在解决什么问题”，是你将来能判断“某个新架构值不值得看”的唯一依据。

〇、本讲要回答的 5 个问题

1. n-gram 语言模型为什么必然失败？它的失败方式是什么？
2. 困惑度 (perplexity) 为什么能当指标？它和 M0 的交叉熵什么关系？
3. RNN/LSTM 解决了什么，又为什么被淘汰？
4. seq2seq + attention (2014) 与 Transformer (2017) 的关键差别在哪？
5. 为什么说“并行性”是 Transformer 取胜的主因，而不是“注意力更聪明”？

一、n-gram：用计数做语言模型

链式法则： $P(w_1 \dots w_T) = \prod_t P(w_t | w_{<t})$ 。n-gram 的近似：只依赖前 $n-1$ 个词， $P(w_t | w_{t-n+1} \dots w_{t-1})$ ，用语料计数估计：

$$P(w_t | w_{t-1}) = \frac{c(w_{t-1}w_t)}{c(w_{t-1})}$$

两个死穴（这就是“为什么历史必然翻篇”）：

死穴	具体表现
稀疏	没见过的组合概率为 0 → 平滑 (add-k / 回退 / Kneser-Ney) 只是打补丁
上下文太短	$n=5$ 时也只能看 4 个词；而“这句话的主语是谁”可能要跨 50 个词

评价指标：困惑度 $PPL = \exp\left(-\frac{1}{T} \sum_t \log P(w_t | w_{<t})\right)$ —— 正是 M0 里交叉熵取指数。PPL 越低，模型对下一个词越不“惊讶”。这个指标一路沿用到今天 (bpb 是它的分词器无关版本)。

二、词向量：从 one-hot 到分布式表示

one-hot 的问题：任意两词的内积都是 0 —— “猫”和“狗”的距离与“猫”和“民主”一样远。语义无从谈起。

word2vec (skip-gram + 负采样)：用一个“给定中心词预测上下文词”的浅层网络，训练出的中间层权重就是词向量。结果是著名的向量算术： $\overrightarrow{king} - \overrightarrow{man} + \overrightarrow{woman} \approx \overrightarrow{queen}$ 。

关键转折：人们发现“预测下一个词”这个任务逼着模型学出语义结构。这正是后来 GPT 路线的思想雏形：语言建模本身就是最好的自监督任务。

三、RNN / LSTM：把“历史”压进一个向量

RNN: $h_t = \tanh(W_h h_{t-1} + W_x x_t)$ ，用 h_{t-1} 携带历史。LSTM：加门控（遗忘门、输入门、输出门）让信息可以“长期保留”。

为什么被淘汰（两个根因，都是工程性的）：

1. **梯度消失/爆炸**: BPTT 要对时间展开求导, 梯度是 $\prod_t \frac{\partial h_t}{\partial h_{t-1}}$ 的连乘。连乘里每个 Jacobian 的谱半径 $<1 \rightarrow$ 指数衰减; $>1 \rightarrow$ 爆炸。这是结构性问题, 不是调参能解决的。
2. **无法并行**: h_t 依赖 h_{t-1} , 必须按时间顺序串行计算。GPU 有上万核心却只能一次算一步 —— 训练吞吐被“时间”卡死。

这是本讲最重要的一句话: 深度学习的历史不是“模型更聪明”的历史, 而是“让更多算力能有效地作用于更多数据”的历史。Transformer 赢在能并行。

四、seq2seq 与注意力 (2014): 注意力的真正起源

seq2seq 结构: 一个 RNN 把源句编码成一个固定长度向量, 另一个 RNN 从它解码出目标句。

瓶颈: 整句信息被压进一个向量 —— 句子越长, 丢得越多。

Bahdanau 注意力 (2014) 的解法: 解码每一步时, 回头扫描编码器的所有隐状态, 算出“当前该关注哪几个位置”, 加权求和作为当前步的上下文。→ 这就是 **QKV 的雏形** (解码器状态=Query, 编码器隐状态=Key/Value)。

关键历史结论: 注意力最初是给 RNN 打补丁的, 不是 Transformer 的胜利品。Transformer 的贡献是: “既然注意力这么好用, 那还要 RNN 干什么?” —— 把 RNN 整个删掉, 只用注意力 + 前馈层, 于是整段序列可以并行计算。

路径长度对比 (理解“为什么更快”的关键):

架构	任意两位置的依赖路径	并行性
RNN	$O(L)$ (要穿过 L 步)	✗ 串行
Transformer	$O(1)$ (一步直达)	✓ 全并行

代价是注意力本身 $O(L^2)$ 的计算量 —— 这是“用算力换并行”的经典交易, 也是后来 flash attention、稀疏注意力、线性注意力等一整条研究线的起点。

五、常见误解

误解	更正
“注意力是 Transformer 发明的”	2014 年 Bahdanau 就用于 seq2seq; 2017 只是去掉 RNN 并改成 self-attention
“self-attention 与 attention 是两回事”	同一机制, 只是 Q/K/V 都来自同一序列
“RNN 不行是因为效果差”	在长依赖上确实差, 但致命的并行性才是被淘汰主因
“PPL 越低模型越强”	只在同分词器、同数据下可比 (见 M2)

六、能立刻跑的实验

```
# 用 n-gram 的视角感受“上下文长度”的代价: 统计 bigram 的稀疏程度
环境/.venv/bin/python - <<'PY'
import collections, re
text = open("上游/assignment1-basics/tests/fixtures/tinystories_sample.txt",
encoding="utf-8").read().lower()
words = re.findall(r"[a-z']+", text)
uni = collections.Counter(words)
```

```

bi = collections.Counter(zip(words, words[1:]))
print(f"词数={len(words)} 不同词={len(uni)} 不同的相邻词对={len(bi)}")
print(f"→ 光是一个小语料，二元组就已经接近词数的 {len(bi)/len(uni):.1f} 倍；三/四元组的稀疏会爆掉")
print(f"→ 换成长度更大的语料，这个比值会继续上升（稀疏问题的定量感受）")
PY

```

七、代码级提问（3–5 道）

1. 【推导】为什么 BPTT 的梯度是 Jacobian 连乘？写出两步的式子，并说明“谱半径 < 1 就衰减”的含义。
2. 【对比】用一句话说清：Transformer 相比 RNN，用掉了什么、换来了什么？
3. 【判断】如果有一天硬件变成“串行极快、并行极慢”，Transformer 还会是主流吗？说理由。
4. 【联系】d2l 里 attention 的示意图和 M3 讲的 QK^T 是一回事吗？把两者对应起来。
5. 【指标】你的 n-gram 模型 PPL=120，另一个模型 PPL=80，但用的是不同分词器。能说后者更强吗？

八、记忆锚

- 隐喻：RNN = “传话游戏”——一句话传到第 50 个人已经面目全非；注意力 = “所有人同时看着原始白板”——谁离得远都不影响看清原文。
- 口诀：“n-gram 死于稀疏、RNN 死于串行、注意力先治 RNN、Transformer 删掉串行”
- 符号位置法：把 $O(L)$ 与 $O(1)$ 并排写在卡片上——左边是“要走 L 步”，右边是“一步直达”；下面是代价 $O(L^2)$ ，提醒你注意力不是免费的。

讲义 M2 · 分词与表示

对应课题：课题 01 BPE 分词器 (W1) | 模块：M2 | 预计 2 个时段 (3–5 小时) 对标材料：CS336 A1 handout (规格权威)· 李沐 d2l ch14.6 子词嵌入 (中文)· HF LLM Course ch2 读法：先读下面的“要回答的问题”，再读对标材料，最后回来看本讲义主干。

〇、本讲要回答的 5 个问题

1. BPE 训练究竟在优化什么？为什么说它“不是词汇表，而是压缩器”？
2. 为什么必须从字节起步？代价是什么？
3. 训练算法为什么容易写成 $O(n^2)$ ？如何做到官方要求的 <1.5 秒？
4. 编码时为什么必须“回放合并历史”，而不能“最长匹配优先”？
5. 同一个模型，换分词器后 loss 不可比——那该用什么指标？

一、压缩视角：BPE 到底在做什么

一句话：给定词表预算 V ，BPE 反复执行“找出当前出现次数最多的相邻 token 对 $\rightarrow$ 合并成一个新 token”。每合并一次，语料的总 token 数减少一点，直到词表填满 V 。

形式化：把语料看成一串字节 $b_1 b_2 \dots b_n$ 。定义当前切分下相邻对 (x, y) 的频次 $c(x, y)$ 。每步选

$$(x^*, y^*) = \arg \max_{(x, y)} c(x, y)$$

把语料中所有该对合并为新 token xy ，并把 (x, y) 记入 merges 列表（顺序即“资历”）。

关键推论 A（压缩率）：合并的收益可以直接量化——每合并一处，总 token 数减 1。所以“BPE 训练过程”就是“贪心压缩过程”，merges 列表本质是一本压缩字典，且按收益从大到小排序。

关键推论 B（为什么 loss 不可跨分词器比较）：token 越少 $\rightarrow$ 每个 token 承载信息越多 $\rightarrow$ 单 token 的交叉熵必然更高。这是记账单位问题，不是模型好坏。所以跨分词器比较必须换成与切分无关的单位：bits-per-byte (bpb)。

这条是新课标里最重要的一条：nanochat 全程用 bpb 而不是 loss 来跨模型比较，课题 03 (scaling law) 和课题 08 (RL 对照) 都会回到这里。

二、为什么必须字节起步（以及代价）

从 Unicode 字符起步的三个具体麻烦：

麻烦	具体例子	后果
词表爆炸且长尾	仅 CJK 就有 9 万余字符，常用仅数千	大部分字符训练不充分，嵌入近乎随机
永远有未登录	新 emoji（每年新增）、生僻字、方言字	无兜底机制 $\rightarrow$ OOV，直接崩
失去字节级共享	“妈”与“吗”共享部首但完全不同字符	学不到字形/字节层面的结构

从字节起步：任何文本 $\rightarrow$ UTF-8 $\rightarrow$ 0–255 的字节序列 $\rightarrow$ 这 256 个 token 永远在词表里。所以字节级 BPE 永远不存在“无法表示的输入”，只可能被切得更碎。这就是“零风险”的含义。

代价（必须知道，因为它是第 1 题的答案关键）：

- 汉字 UTF-8 占 3 字节；英文常用词常被合并成 1 token，而中文常需 1.5–2 个 token 表示 1 个汉字（取决于词表大小与语料配比） $\rightarrow$ 中文的“每字成本”高于英文

- 序列变长 → 注意力计算量 ($O(L^2)$) 上升 → 中文推理成本更高

三、训练算法：从 $O(n^2)$ 到官方要求的 <1.5 秒

朴素想法的三个陷阱（都会导致超时或超内存）：

1. 每步重扫全语料统计相邻对 → $O(V \cdot n)$, $V=500$ 、 $n \approx$ 百万级就是灾难
2. 用字符串做合并 (`text.replace("ab", "ab_new")`) → 每次 $O(n)$ 拷贝 + 无法区分“新合并的 token”与“原始字符”
3. 把语料整体读入内存再展开成 token 列表 → 5MB 语料 + 1MB 配额的内存测试必挂

正确方向的三个要点（方向，不是实现）：

- 词频级统计而非字符级：同一种“词”（如 the）在语料里重复百万次，只需统计一次它的结构，再按词频加权
- 增量更新：一次合并只影响“包含该对的相邻位置”，其余的对频次不变 → 只需局部更新
- 优先队列/堆：快速取出当前最高频的对，而不是每步全量排序

官方参考实现 0.38 秒，“玩具实现”约 3 秒——这 8 倍差距就在上面三点里。

四、编码：回放合并历史，而不是最长匹配

编码时我们没有语料统计，只有 merges 列表（一个有序的“合并历史”）。正确做法：

把文本按 UTF-8 切成字节数组

loop:

找出当前序列中所有相邻对里，在 merges 中“排名最靠前”（资历最老）的那一对

合并它

直到没有任何相邻对出现在 merges 中

为什么不是“最长匹配优先”？因为训练时每一次合并都遵循“最老优先”的规则；若编码用另一套规则，同一个子串在不同上下文会被切成不同 token，与 tiktoken/GPT-2 的逐 id 对齐测试必然失败。BPE 的编码本质是回放训练时的决策序列。

特殊 token 是另一套优先级：它们不参与合并，且在编码时先于普通文本被匹配。注意两个官方测试点名的情况：重叠的特殊 token（如 a 与 ab 同时存在）与尾随换行 (`<|endoftext|>\n`) ——这类边界的处理顺序会直接决定对错。

五、接口设计决定内存上界（`encode` vs `encode_iterable`）

官方测试里有一个耐人寻味的设置：

方法	官方态度
<code>encode(text)</code>	被 xfail 标记：“encode 预期会超过 1MB 配额”——超了也原谅
<code>encode_iterable(iterable)</code>	用 RLIMIT_AS 硬限 1MB，读 5MB 语料——超了就失败

为什么这不是双标：接口的输入形态决定了它的内存下界。`encode(str)` 的输入本身就是一整段字符串，它已经在内存里了——上界由输入决定，实现者无能为力。`encode_iterable(iterable)` 的契约是流式：输入是迭代器，实现就必须逐块处理、边读边 yield。任何 `list(iterable)` / `"".join(iterable)` / 先聚合再编码的写法，都违背了接口语义。

这是软件工程里最漂亮的一课：接口不只是名字，它是**约束**。同一个算法，换个接口，内存上界就从“输入大小”降到“块大小”。这个思想在课题 05（KV cache）、课题 07（数据流式清洗）、课题 10（长程 Agent 的记忆压缩）都会反复出现。

六、词表大小的三方权衡（第 1、4 题的答案框架）

词表从 V 扩到 4V，同时影响三件事：

受影响项	方向	机理
嵌入/输出层参数量	↑ 约 4 倍	参数 $\propto V \cdot d$ （若权重共享则 1 份，否则 2 份）
同文本序列长度 L	↓	更多常见片段被合并 $\rightarrow$ token 数下降
每 token 的注意力计算	↓ 约 L^2 下降	注意力是 $O(L^2 \cdot d)$
罕见 token 的训练充分度	↓	长尾 token 出现次数少，嵌入学不好

净效果**不是单调的**：太小 $\rightarrow$ 序列太长、算力烧在长度上；太大 $\rightarrow$ 参数浪费 + 长尾欠训练。经验上 32k–128k 是常见区间（GPT-2 用 50257，现代模型多在 100k 量级，且对多语言友好）。

中文特例：词表若中文覆盖不足，一个汉字可能被拆成 3 个字节 token，**中文成本直接翻 2–3 倍**——这是国产模型普遍重视中文词表预算的原因。

七、常见误解（每条配反例）

误解	反例
“一个 token 就是一个词”	<code>_transformer</code> 可能是 1 个 token，妈 可能是 3 个 token（UTF-8 三字节）
“分词只是预处理，不影响模型能力”	词表决定序列长度 $\rightarrow$ 决定有效上下文与算力分配 $\rightarrow$ 直接影响能学到什么
“编码用最匹配更聪明”	会与训练决策不一致 $\rightarrow$ 与 tiktoken 逐 id 对齐测试直接失败
“词表越大越好”	长尾 token 训练不足 + 嵌入层参数浪费；存在经验最优
“decode 一定无损”	只有在 完整 utf-8 序列 上才无损；单 token 解码可能是半个字符 (🐉)

八、能立刻跑的实验（讲完就做，知行咬合点）

```
# ① 跑判分基线（现在应当是 27 failed + 1 xfailed, 全部因缺手写文件）
环境/judge/judge.sh 01 --quick

# ② 看官方参考 merges 长什么样（前 20 条）——体会“资历顺序”的直觉
head -20 上游/assignment1-basics/tests/fixtures/train-bpe-reference-merges.txt

# ③ 亲眼看字节：一个汉字到底占几个 token（用官方 fixture 的 GPT-2 词表）
环境/.venv/bin/python - <<'PY'
import tiktoken, os
os.environ["TIKTOKEN_CACHE_DIR"] = "环境/tiktoken-cache"
enc = tiktoken.get_encoding("gpt2")
for s in ["hello world", "你好", "你好，世界！", "🐉"]:
    ids = enc.encode(s)
    print(f"{s!r:16}  $\rightarrow$  {len(ids)} 个 token {enc.decode([i] for i in ids)}")
print("<math>\rightarrow</math> 汉字/emoji 被切成多个字节 token 的现象，一眼可见")
PY
```

九、代码级提问（3-5 道，先写预测再动手）

1. **【改坏预测】** 把训练里“并列最高频时任意取一个”改成“取字节序最小的一对”，`test_train_bpe` 会过吗？`test_train_bpe_speed` 呢？**先写预测和理由。**
2. **【诊断】** 你的实现通过了往返一致性测试，但 `test_train_bpe` 的 merges 比对失败。请按顺序列出你会检查的 4 件事（提示方向：预处理、并列规则、特殊 token、词表初值）。
3. **【代价估算】** 同一句话，词表从 32k 换到 128k：①嵌入参数量变几倍 ②序列长度大致变几成 ③**bpb** 会变吗？为什么？
4. **【接口】** 若把 `encode_iterable` 写成“先 `list(iterable)` 再拼接”，哪个测试会挂？挂的形式是断言失败，还是进程被直接杀掉？（想清楚 `RLIMIT_AS` 的作用方式）
5. **【迁移】** 课题 05 要写 KV cache，它会遇到和“`encode` vs `encode_iterable`”同构的内存问题吗？说说你的类比。

十、记忆锚

- 隐喻：BPE = “贪心的拼音缩写机”——不停找最常一起出现的两段，把它们缩成一个新记号，缩到最后词表满了；字典按先来后到排资历。
- 口诀：“字节兜底、频次贪心、老手优先、流式省内存”（对应：字节起步 / 训练目标 / 编码回放 / 接口约束）
- 符号位置法： V （词表）在上，撑着两件事：下面是 参数量 $\propto V \cdot d$ ，右边是 序列长度 $L \downarrow$ ；三者写成一个三角形，随时能想起“扩词表 = 拿参数换长度”。

十一、本讲与后续的接口（预告）

- **M3:** 序列长度 L 怎么进入注意力计算量 ($O(L^2)$) ——词表选择的代价在这一讲兑现
- **M4:** bpb 作为跨规模比较单位——scaling law 的纵轴
- **M5:** `encode_iterable` 的流式思想 → KV cache 的“不重算历史”
- **课程板块:** 本讲直接对应课程大纲的“大语言模型”入门，也是大论文“方法”章的第一段素材

讲义 M3 · Transformer 内部机制

对应课题：课题 02 Transformer 骨架 (W2-W3) | 模块：M3 | 预计 2-3 个时段 对标材料：CS336 lecture 03-04 + A1 handout (模型部分) · rasbt 《LLMs-from-scratch》ch3-ch4 (附录含 GQA/MLA/MoE) 这是全学科最核心的一讲：唐杰作业①的“从零写 Transformer”就落在这里。

〇、本讲要回答的 5 个问题

1. 注意力到底在算什么？为什么缩放因子是 $\frac{1}{\sqrt{d_k}}$ ？
2. 为什么要多头？多头比单头强在哪、代价是什么？
3. RoPE 凭什么比可学习位置嵌入更能外推？
4. RMSNorm 凭什么能替代 LayerNorm？pre-norm 与 post-norm 差在哪？
5. 一个 d 宽、 L 层、词表 V 的 Transformer 有多少参数？KV cache 有多大？

一、注意力：加权检索 + 为什么缩放 $\frac{1}{\sqrt{d_k}}$

计算式 (先记住形状，再看含义)：

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

其中 $Q, K, V \in \mathbb{R}^{L \times d_k}$ ， M 是因果掩码 (下三角为 0、其余 $-\infty$)。

含义：每个位置发出一个查询 q ，去和所有位置的键 k 做内积 (相似度)，softmax 归一化成权重，再对值 v 做加权平均。它就是“按相似度检索并汇总”。

为什么除以 $\sqrt{d_k}$ ：设 q, k 各分量独立、均值 0 方差 1，则内积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的方差是 d_k (方差可加)。 d_k 越大，内积的取值范围越宽 $\rightarrow$ softmax 越容易“饱和” (一个位置拿走几乎全部权重) $\rightarrow$ 梯度趋近 0 $\rightarrow$ 训练不动。除以 $\sqrt{d_k}$ 把内积的方差重新拉回 1 量级。这是“训练稳定性”的第一课。

因果掩码：语言模型不能看未来。做法是在 softmax 之前给未来位置加 $-\infty$ (实现上常用 -10^9 之类的大负数，而不是 0) —— 因为在 softmax 之后置零会改变概率归一化 (等价于把已分配的概率丢掉，剩下的位置没有重新归一)。

在代码里长什么样：手写/里会有 `scaled_dot_product_attention` 与 `multihead_self_attention` (含 causal 与 RoPE 两个变体)；上游对照 `nanochat/gpt.py` 的 `CausalSelfAttention`。注意接口而非实现：CS336 的 `tests/adapters.py` 规定了你要暴露的函数签名。

二、多头：拆开看不同关系，但总算力不变

做法：把 d 维拆成 h 份、每份 $d_k = \frac{d}{h}$ ，各自独立做注意力，再拼回来过一个输出投影 W_O 。

关键理解 (常被误解)：多头不增加注意力的总计算量——单个头用满 d 维时， $QK^\top$ 是 $L^2 d$ ； h 个头各 $\frac{d}{h}$ 维合计仍是 $L^2 d$ 。多头的收益在于归纳偏置：不同的头可以在不同子空间里学不同的关系 (有的学“上一个词是谁”，有的学“括号配对”，有的学“句子边界”)，比单一头更灵活。

变体 (现代模型常用)：

变体	做法	动机
MQA	所有头共享一份 K/V	KV cache 减小 h 倍，但质量略降

变体	做法	动机
GQA	头分组，组内共享 K/V（如 8 组）	折中：cache 减小、质量几乎不掉 ← 现代主流
MLA	把 K/V 压成低维潜变量再解压	更激进，DeepSeek 系

为什么都在动 K/V：推理时的瓶颈是 KV cache 的显存（见 § 五），它随 层数 × 序列长度 × 头数 线性增长，与“生成速度”直接相关。这是“架构为推理服务”的典型例子。

三、位置编码：从“贴标签”到“转角度”

问题：注意力的内积是置换不变的——把 token 顺序打乱，结果不变。所以必须注入位置信息。

三代方案：

方案	做法	缺陷
可学习绝对嵌入	学一个 $L_{\max} \times d$ 的查表	不能外推：训练 2048，推理 4096 时后一半没学过
正弦绝对编码（原论文）	$\frac{\sin}{\cos}$ 固定频率	可外推性弱，实践中被 RoPE 取代
RoPE（旋转位置编码）	把 q, k 按二维子空间旋转角度 $\theta \cdot m$	内积只依赖相对距离 $m - n$ ；外推性好

RoPE 的核心恒等式（这是它存在的全部理由）：

$$\langle R_m q, R_n k \rangle = \langle q, R_{n-m} k \rangle$$

即：在内积里只出现“位置差”，绝对位置被“旋转”吸收掉了。推导要点：旋转矩阵是正交阵， $R_m^\top R_n = R_{n-m}$ 。

频率分配：第 i 对维度的角频率 $\theta_i = 10000^{-2\frac{i}{d}}$ ——低维（ i 小）转得快（管近距离），高维转得慢（管远距离）。“10000”这个底数是“能覆盖多长上下文”的旋钮，这就是所谓 YaRN / NTK 插值等长上下文外推技巧的入口。

在代码里长什么样：手写/ 的 rope 与前缀 rope 的多头注意力；CS336 有单独的 run_rope 测试点。

四、归一化与残差：让 100 层能训起来的两件套

LayerNorm：对每个 token 的 d 维向量做 $\frac{x-\mu}{\sqrt{\sigma^2+\epsilon}}$ ，再缩放和平移。RMSNorm：只做 $\frac{x}{\sqrt{\text{mean}(x^2)+\epsilon}}$ ，再乘一个可学缩放 g 。

- 省掉了减均值与平移：更便宜（少一次归约、少一组参数）
- 效果相当：真正起作用的是尺度不变性（分母归一化），而中心化贡献有限
- 现代模型（LLaMA/Qwen/GLM 系）默认 RMSNorm

残差连接： $x \leftarrow x + f(\text{norm}(x))$ （pre-norm）。它防的是梯度消失：反向传播时，加法的 Jacobian 是单位阵，于是梯度有一条恒等通路可以直达浅层。没有它，几十层的网络基本训不动。

pre-norm vs post-norm（原论文是 post-norm）：

	形式	性质
post-norm	$\text{norm}(x + f(x))$	原版；深了需要小心 warmup，容易发散
pre-norm	$x + f(\text{norm}(x))$	训练稳，是现代默认；代价是“深度”的表示能力稍弱

课题 02 会做“去掉残差”的改坏实验：你会亲眼看到 loss 从“平稳下降”变成“几乎不动”。

五、FFN、参数量与 KV cache（把结构换算成资源）

FFN 的演化：两层线性 + 激活。原版 $d_{ff} = 4d$ 配 ReLU。SwiGLU 用三个矩阵：Swish(xW_1) $\odot$ (xW_3) 再过 W_2 ；为保持参数量不变，取 $d_{ff} \approx \frac{8}{3}d$ （3 个矩阵 $\times 8/3 \approx 4$ 个矩阵 $\times d$ 的等价参数量）。

单层参数量（ d 宽，标准 MHA）：

- 注意力： W_Q, W_K, W_V, W_O 各 $d \times d \rightarrow 4d^2$
- MLP (SwiGLU, $d_{ff} = \frac{8}{3}d$)： $3 \cdot d \cdot \frac{8}{3}d = 8d^2$
- 两处 RMSNorm 缩放： $2d$ （可忽略）
- 合计 $\approx 12d^2$ 每层 $\rightarrow$ 全模型 $\approx 12Ld^2 + Vd$ （权重共享时）

验算习惯：拿到任何模型的配置（如 $d=768, L=12, V=50257$ ），先心算 $12 \times 12 \times 768^2 \approx 85M$ ，加嵌入 $50257 \times 768 \approx 39M$ （共享则算 1 份） $\approx 124M$ —— 与 GPT-2 的 124M 吻合。这种“能不能心算”是 M3 的验收标志之一。

KV cache 显存（推理，重点）：

$$\text{bytes} = 2 \times L_{\text{layers}} \times L_{\text{seq}} \times n_{kv} \times d_k \times \text{bytes/elem}$$

（乘 2 是因为 K 和 V）。用 GQA 后 n_{kv} 从 h 降到组数，cache 直接按比例缩小——这就是 GQA 流行的真正原因。课题 05 会实测这个数字。

六、常见误解（每条配反例）

误解	反例 / 更正
“注意力=可解释的检索”	注意力权重高 $\neq$ 因果重要（已有大量反例）；它是 加权平均的权重 ，别过度解读
“多头=多算”	总 FLOPs 不变（§ 二），只是分成子空间
“位置编码只是加个向量”	RoPE 是 旋转 Q/K ，不是加到输入上；且它只影响内积的相位
“RMSNorm 是 LayerNorm 的近似”	它 省略了中心化 ，不是近似；在很多模型上效果更好且更省
“参数量大就一定强”	同样算力下，参数与数据要按 scaling law 配比（课题 03 的正题）
“因果 mask 随便加个大负数就行”	必须在 softmax 之前 ；加在之后会破坏归一化

七、能立刻跑的实验与改坏实验（课题 02 的里程碑）

基础验证（跑通即达里程碑 M1-M3）：

```
bash 环境/bootstrap_cpu.sh --with-torch # 课题02 起需要 torch
环境/judge/judge.sh 02 --quick # 形状与数值测试（判分器按需接入）
```

三组改坏实验（先写预测，再跑，再记录——这是本讲的核心作业）：

#	改哪里	先预测什么	预期观察（跑完记得回来对照）
1	去掉因果 mask(让所有位置互相可见)	训练 loss 会大幅下降，但采样出来的文本“语法很好、内容胡说”	典型“作弊”：loss 好看，能力为零
2	RoPE 换成可学习绝对嵌入	训练集内正常，超出训练长度立刻崩	外推性丧失；正是课程要“用实验看见”的性质
3	去掉残差连接	浅层（ ≤ 4 层）还能训，深了就完全不收敛	梯度消失的实证

记录要求：每条都要有“预测 $\rightarrow$ 现象 $\rightarrow$ 解释”三段，落进 课题 02/证据/。预测错是资产，不是污点。

八、代码级提问 (3-5 道)

1. 【推导】用“方差可加”说明： $q \cdot k$ 的方差是 d_k ，并解释为何要除以 $\sqrt{d_k}$ 而不是 d_k 。
2. 【心算】配置 $d = 1024, L = 24, V = 100000$ ，权重共享。估算参数量（结果用 G/M 表示），并拆出嵌入层占比。
3. 【改坏预测】把 causal mask 从“softmax 之前”挪到“softmax 之后”（对已归一化权重置零），训练会发生什么？给出两个可能的现象并说明理由。
4. 【联系】用 M0 的 logsumexp 解释：为什么 flash attention 的分块计算能保持数值稳定？
5. 【代价】序列长度从 2k 增到 32k：①注意力计算量变几倍 ②KV cache 变几倍 ③哪个先撞墙？

九、记忆锚

- 隐喻：注意力 = “会议室里的自由讨论”——每人 (query) 挑自己关心的话题 (key)，按关心程度 (softmax 权重) 听别人 (value) 说话；因果 mask = “不许听后面发言的人”；多头 = 同时开 h 个并行分会场；RoPE = 每个人的座位号刻在朝向上，所以只关心“相隔几个座位”。
- 口诀：“缩放防饱和、掩码在软前、多头不减算、旋转只认距、残差是命脉、归一挑 RMS”
- 符号位置法： $\frac{QK^T}{\sqrt{d_k}}$ —— 把 $\sqrt{d_k}$ 写在分数线下方，心里念“维度越高，内积越炸，除以根号拉回来”。

讲义 M4-M9 • 展开大纲（按周写完整版）

说明：本文件是 M4-M9 的**骨架大纲**（核心问题 + 关键概念 + 必读 + 实验 + 坑），每周讲课前我会把当周讲义**展开成完整版**（新增 M4-训练.md 等文件），不留空讲义占位。这样做的理由：这些模块严重依赖课题 04 起的**真实实验数据**，提前写死会出现“讲义与实测不符”。

M4 • 训练：数据、优化、系统（W3-W9）

要回答的问题：为什么同一个模型有人训得动有人训崩？scaling law 怎么指导算力分配？多卡为什么不是线性加速？

关键概念：预训练目标与数据配比 | 过滤与去重 | AdamW（解耦权重衰减的意义）| Muon | LR schedule（warmup + cosine）| 梯度裁剪 | 混合精度（bf16 vs fp16 vs fp32, GradScaler 何时必需）| 梯度累积 | DDP / FSDP / TP / PP | Chinchilla 的 compute-optimal | `--target_flops` 与“固定算力预算下的最优 (N, D)”

必读：CS336 lecture 05-11 + A2/A3 handout · nanochat/optim.py、base_train.py、runs/scaling_laws.sh · d2l ch11-ch12

实验：手写 AdamW 与调度 → 单卡固定算力下的 depth sweep（d4/d6/d8/d10）→ 拟合幂律 → **先写死外推预测** → 跑 d12 验证（打脸链路）

坑：把“loss 不降”当数据问题（常是 LR/初始化/精度）| 把 DDP 当免费加速（PCIe 下扩展效率可能只有 1.3-1.5×）| T4 无 bf16（必须 fp16 + GradScaler）| 显存不足先怀疑 batch 与精度，再怀疑硬件

M5 • 推理与部署（W5-W8）

要回答的问题：为什么生成必须逐 token？KV cache 省了什么、代价是什么？量化掉多少点？吞吐与延迟的取舍？

关键概念：自回归解码 | **KV cache**（显存公式见 M3 § 五）| continuous batching | PagedAttention | 采样策略（temperature / top-p / top-k / repetition penalty）| 量化（int8/int4/GPTQ/AWQ）| 投机解码 | 每 token 成本

必读：nanochat/engine.py（KV cache 实现）+ infer_bench.py · CS336 lecture 10 · rasbt《Reasoning》ch2

实验：从零写带 KV cache 的生成 → 测有/无 cache 的耗时与显存 → 贪心 vs top-p 对照 → batch size 对吞吐的影响

坑：以为“慢是因为模型大”（常是没批处理/没 cache）| 以为量化无损 | 忽略 prefill 与 decode 两个阶段特性完全不同（前者算力密集，后者显存带宽密集）

M6 • 后训练：从“会说话”到“会做事”（W10-W12）

要回答的问题：SFT / DPO / RLVR 各补什么能力？为什么 RLVR 提数学/代码却可能伤通用能力？reward hacking 长什么样？

关键概念：指令微调与 chat template | **损失掩码**（只算 answer 部分）| 奖励模型 | PPO 的四个模型 | DPO 的隐式奖励（推导：为什么可以不要 reward model）| **GRPO 的组内归一化优势** | KL 正则 | 拒绝采样 | 蒸馏与合成数据 | 过度优化（over-optimization）

必读: CS336 A5 (含 tests/test_grpo.py、GSPO/MaxRL) · rasbt《Reasoning from Scratch》ch3–ch8 · Nathan Lambert《RLHF Book》(免费在线, 17 章) · minimind/trainer/train_grpo.py · nano-aha-moment (单文件)

实验: 同基座做 SFT / DPO / GRPO 三变体 → 同一评测口径对照 → 手写 GRPO 损失并逐项注释 → **制造一次 reward hacking 并记录**

坑: 以为 DPO “更简单所以更好” | 以为 RL 一定比 SFT 强 | **把训练 reward 上升当能力提升** (不做 held-out 评测就是自欺) | KL 系数设错导致模型 “复读训练集答案”

M7 · 评测：把“感觉变好了”变成“确实变好了” (W9–W12)

要回答的问题: benchmark 分数可信吗？污染怎么判断？为什么必须固定口径？方差有多大？

关键概念: bpb / PPL / CORE | 下游基准 (GSM8K / MATH-500 / MMLU / HumanEval) | pass@k | 评测污染 | few-shot 与 prompt 敏感性 | 方差与误差棒 | LLM-as-judge 的偏置 | 可验证奖励 vs 学习型奖励

必读: rasbt ch3 (先建评测再上技术——全书教学法基石) · nanochat/core_eval.py (单文件 CORE) · CS336 lecture 12 · Berkeley Agentic AI MOOC 的 “Agent Evaluation” 讲

实验: 写一个数学题判定器 (解析 + 数值比对 + 容错) → 测它在 200 题上的准确率与**假阳性** → 给两个模型跑同一套评测并给误差棒

坑: 只看一个 benchmark 下结论 | 忽略 prompt 格式造成的巨大差异 | 把测试集当验证集反复调 | 只报均值不报方差

M8 · 智能体：工具、记忆、harness、长程任务 (W13–W14)

要回答的问题: 从“回答问题”到“完成任务”缺什么？为什么 harness 比模型本身更决定成败？self-judge 何时有效、何时自欺？

关键概念: 工具/函数调用 | ReAct 循环 (Thought-Action-Observation) | **context engineering** (李宏毅与 Karpathy 都专讲) | 记忆与压缩 | harness 设计 (沙箱、权限、反馈信号) | 长程任务的误差累积 | 多智能体 | self-judge / self-refine 的边界 | 可验证环境

必读: 李宏毅《Context Engineering》专讲 · Berkeley LLM Agents / Agentic AI MOOC (含 “Post-Training Verifiable Agents”) · HF Agents Course (三框架 + 公开排行榜) · rasbt ch5 (self-refinement) · PrimeIntellect-ai/verifiers · terminal-bench

实验: 手写最小 ReAct 循环 (不用框架) → 给两个工具 (计算器 + 代码执行) → 记录失败轨迹并分类 → 加 self-judge 做消融

坑: 以为瓶颈在模型智力 (常在你的工具接口与反馈信号) | 以为 self-judge 免费提升 (**它会放大原有错误**) | 上下文不做压缩导致长任务后段 “忘记目标”

M9 · 前沿与安全 (W15–W16 选题制)

要回答的问题: MoE / 长上下文 / 多模态改变了什么？对齐与安全是“加一层”还是“改训练目标”？“AI 训 AI”可行到什么程度？

关键概念: MoE 稀疏激活 (总参数 vs 激活参数) | 长上下文外推 | 多模态对齐 | 可解释性 | 红队与越狱 | 幻觉的成因 | 合成数据闭环 | self-play | 灾难性遗忘 | 模型合并

必读: CS336 lecture 04 (MoE) · opendilab/awesome-RLVR · LeapLabTHU/limit-of-RLVR · 论文精读清单联动

实验（三选一）：合成数据自训练一轮 / 拒绝采样提升数学 / self-play 生成题目 → 做最小闭环 + **明确写出这个方向的 no-go**

坑：把“前沿”当必须立刻掌握（正确姿势是**保持索引能力 + 每季度更新一次本模块**） | 只看结论不看评测口径 | 把“能跑通”当成“有结论”

展开时机表

讲义文件	需要先有	预计展开时间
M4-训练.md	课题 02 收口（能算 FLOPs/显存）	W3
M5-推理.md	课题 04 首轮训练成功（有模型可推理）	W5
M7-评测.md	有第一个模型输出	W9
M6-后训练.md	课题 07 数据管线 + 评测口径固定	W10
M8-智能体.md	有 SFT 后的模型	W13
M9-前沿.md	全部证据齐备	W15